Career Readiness Advisor

Project Analysis and Design

UML model · use case · activity · sequence · database · class · ER diagrams

17 October 2026

The Career Readiness Advisor is an iOS-styled Flutter app over a stateless FastAPI service. It compares a student's completed courses, and any skills they report from outside class, against what 82 Saudi job postings across eight Information Systems careers ask for. It returns a demand-weighted readiness figure, the missing skills ranked by demand, and the university and online courses that close them. Department staff see the same catalog read against the market.

Every diagram below is drawn from the code and data as they stand on 17 October 2026: names, fields, endpoints and counts are the real ones.

Contents

- 1. UML model: component, deployment and package diagrams
- 2. Use case diagrams and use case descriptions
- 3. Activity diagrams
- 4. Sequence diagrams
- 5. Database diagram
- 6. Class diagrams
- 7. ER diagram

1. UML model

The system has three parts: the iPhone app, a FastAPI service, and the skill-gap engine it wraps. The app never touches the engine or its data files directly: everything goes through seven REST endpoints, and the two that read a student's profile store nothing. The three views show the parts and their dependencies (component), where each part runs (deployment), and how the app's code is organised (package).

1a. Component diagram

The app is layered: screens use Riverpod state, state uses repositories, and repositories call the API over HTTP. The API builds the engine state once at start-up from six data files, and only loads the embedding model when its cache misses.

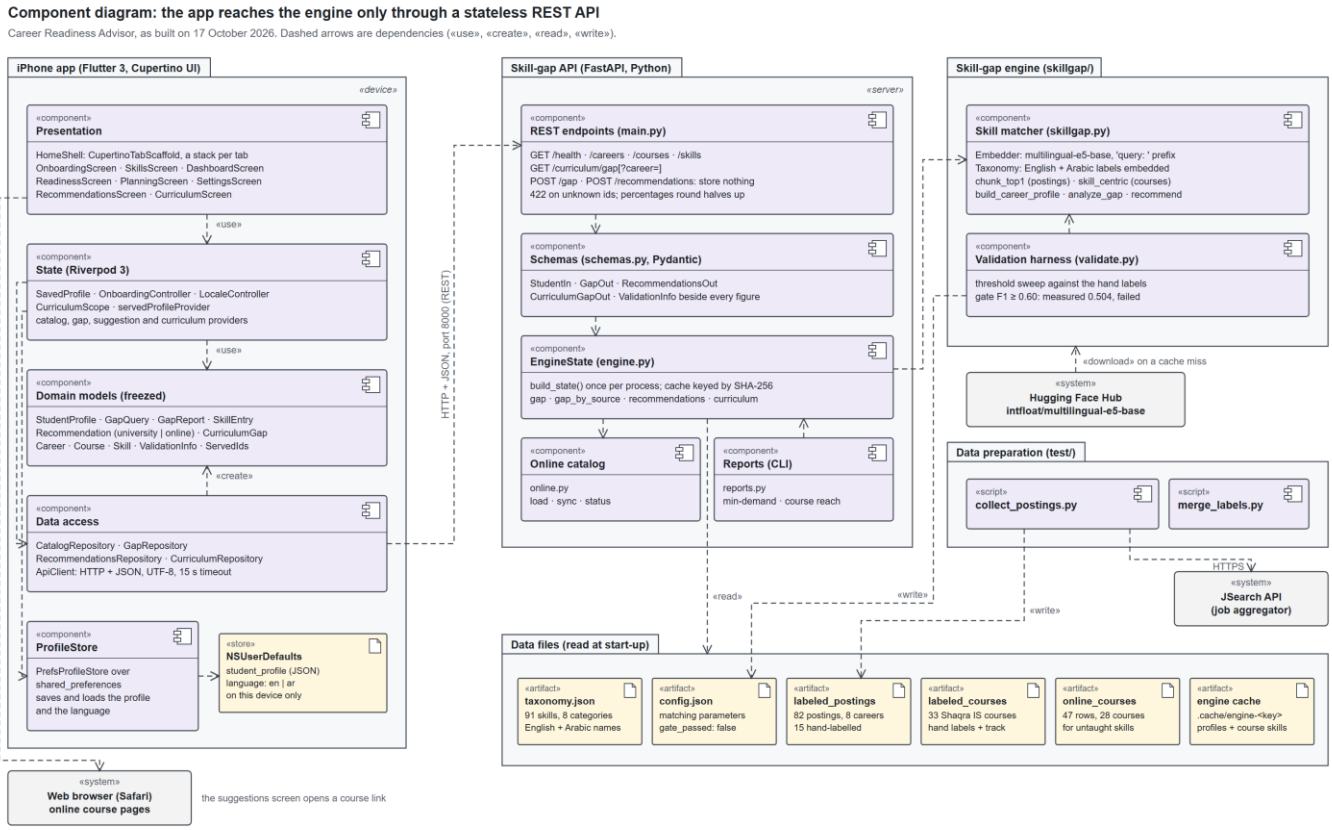


Figure 1. Component diagram of the app, the API, the engine and the data files

1b. Deployment diagram

The app runs on an iPhone or the iOS Simulator and keeps the profile in the phone's own storage. The API runs as one Python process on the development machine, on port 8000. The outside services are used only at build time, at collection time, or when a student opens an online course.

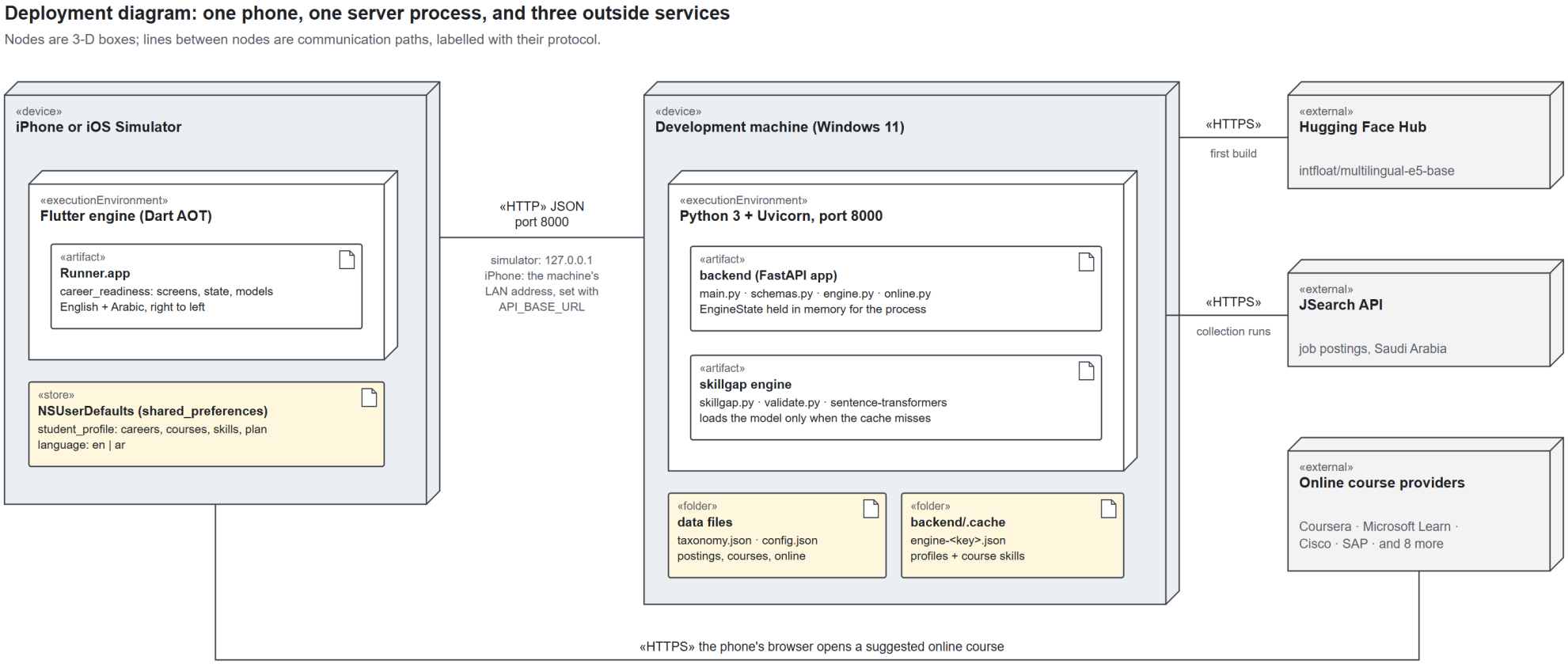


Figure 2. Deployment diagram of the phone, the server and the outside services

1c. Package diagram

The app's code is split into eight feature packages over a shared core. Each feature keeps only the layers it needs: home and settings are screens only, and planning reuses the gap call instead of having its own data layer.

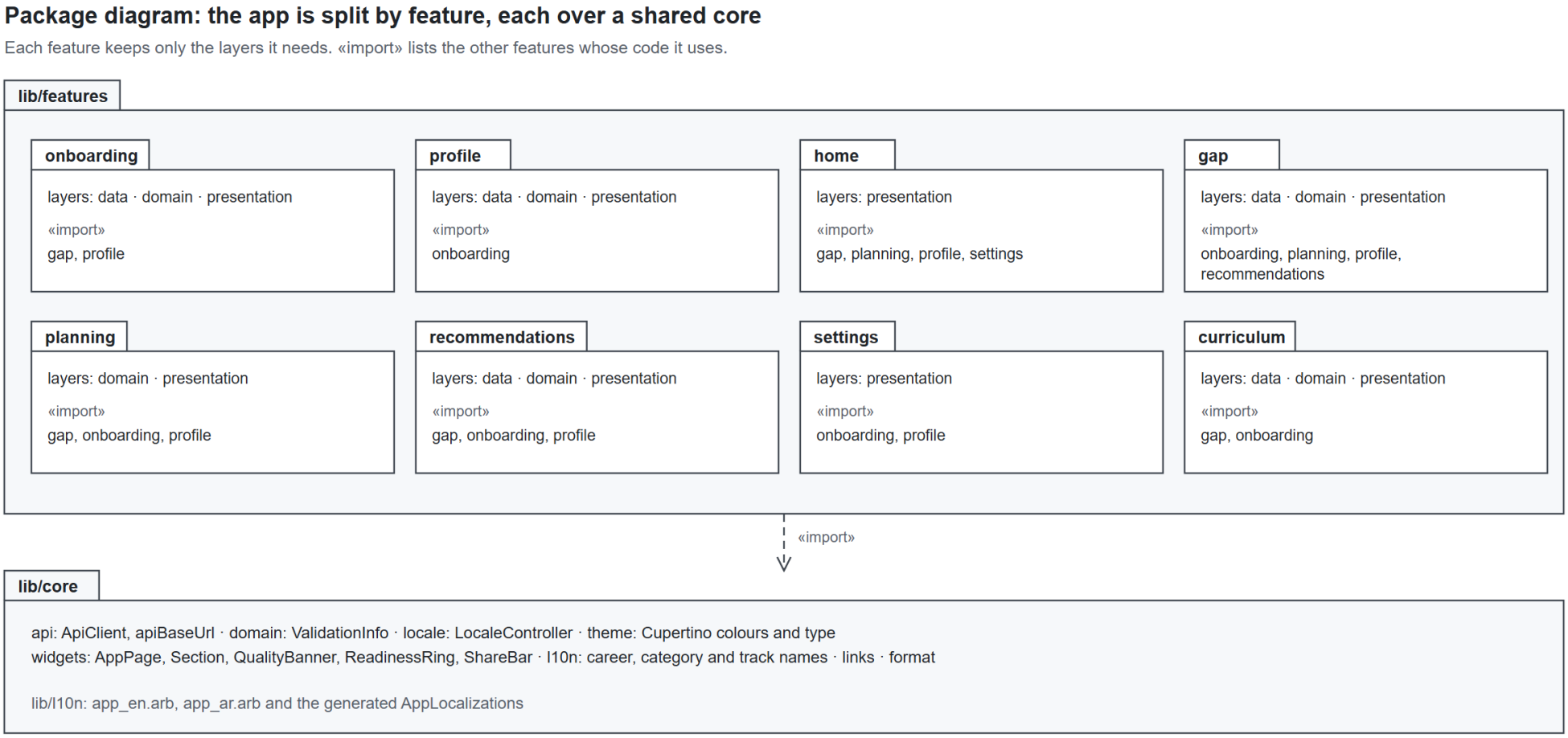


Figure 3. Package diagram of the app's feature packages

2. Use case diagrams

Two kinds of user work in the app: students and department staff. Both are app users, so both can switch language. A third actor, the researcher, works only with the offline tools that prepare the data the app reads.

2a. App use cases

Creating and editing a profile share one form. Comparing careers only happens when a profile names two or three careers, and opening an online course hands the student to the browser.

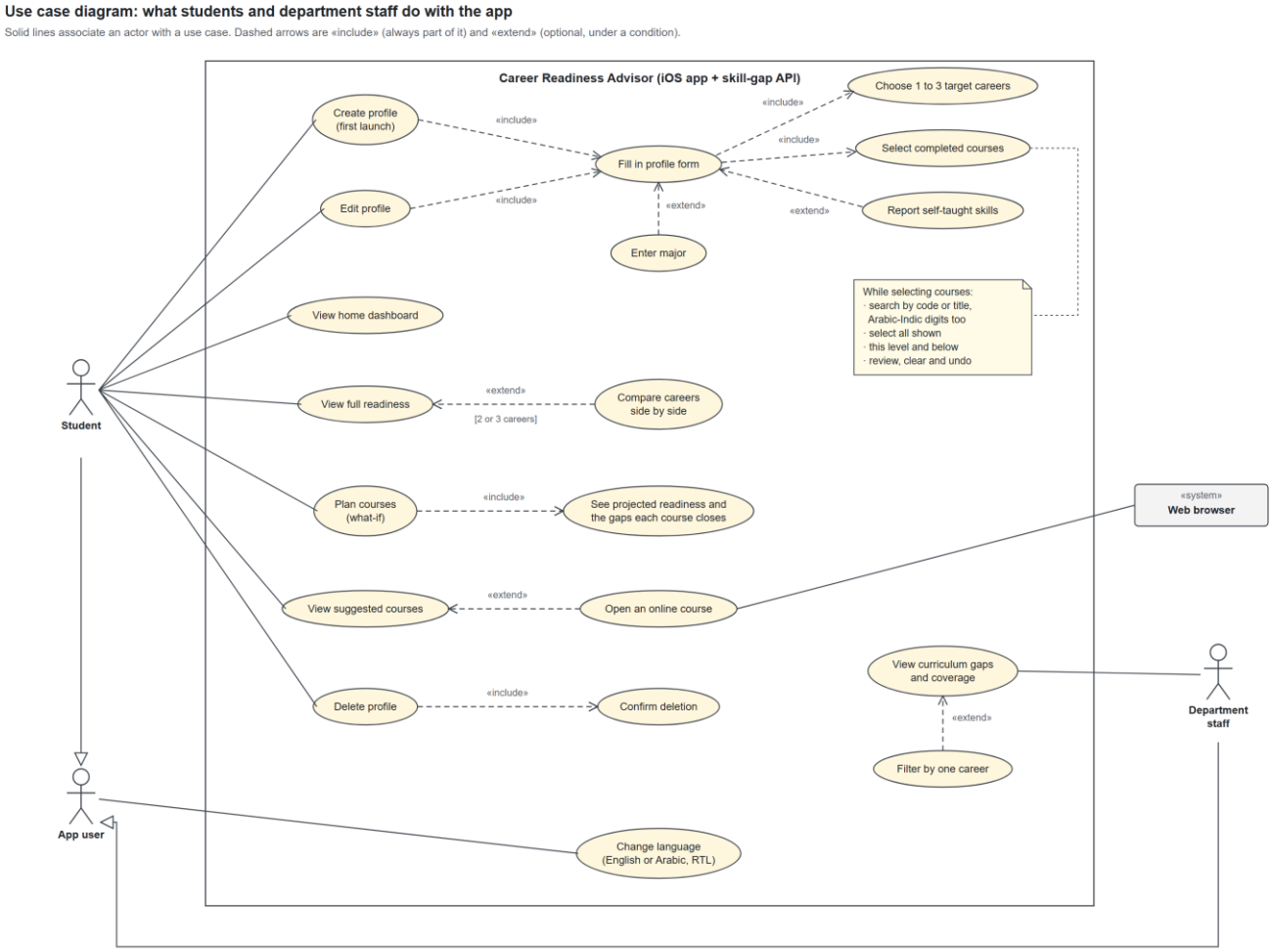


Figure 4. Use case diagram for students and department staff

2b. Data-preparation use cases

The researcher collects and labels the data, measures the matcher, and starts the API. The embedding model is loaded by the validation sweep, by the course-reach report's --explain option, and when the engine's cache misses.

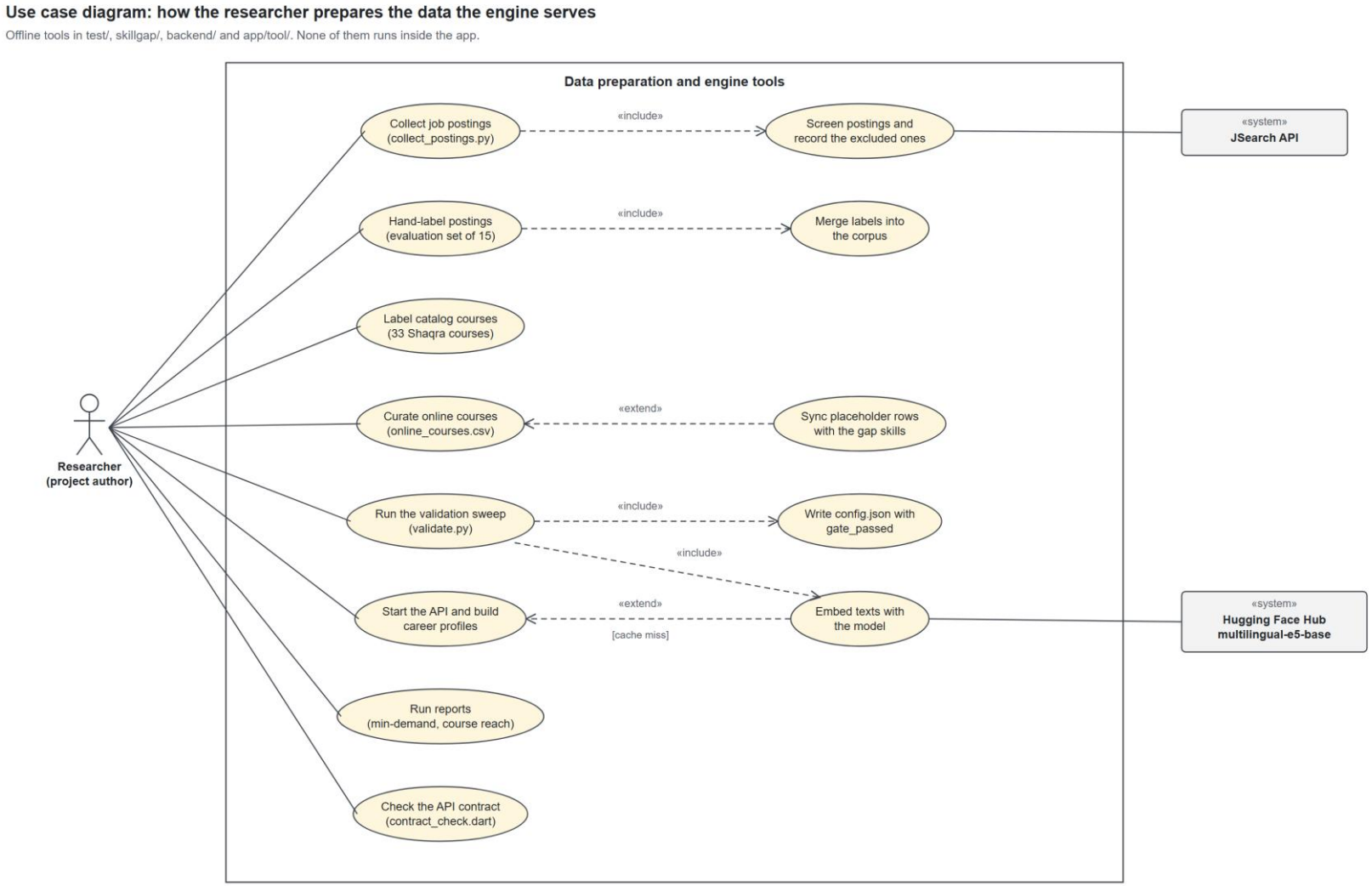


Figure 5. Use case diagram for the researcher's data-preparation tools

2c. Use case descriptions

The five use cases that carry the app's purpose, written out.

UC-1 Create profile

Field	Description
Actor	Student
Precondition	No profile is saved on this device
Main flow	1) The app opens on “Your profile” and loads the careers and courses from the API. 2) The student picks 1 to 3 target careers and may type a major. 3) The student ticks completed courses, with search, “select all shown”, a level shortcut, or review and clear. 4) “Next: other skills” opens the skills step, where the student may tick skills learned outside class; skills the chosen courses teach are marked and cannot be ticked. 5) “See my readiness” saves the profile on the device and opens the home dashboard.
Alternate flows	1a) The API is unreachable: an error with “Try again”. 2a) A fourth career is refused. 5a) The save fails: the skills step says so and keeps the draft.
Postcondition	The profile is saved on the device, and onboarding does not run again while it exists

UC-2 View full readiness

Field	Description
Actor	Student
Precondition	A profile is saved
Main flow	1) From home, the student taps “Full readiness”. 2) The app sends the profile, read against the main career, to POST /gap. 3) The screen shows the quality flag, the readiness percentage, covered skills split into “from courses” and “self-reported”, and missing skills ranked by their share of postings. 4) Where self-reported skills change the figure, the courses-only figure stands beside it.
Extension	With 2 or 3 careers, they stand side by side with score, covered and missing counts, and posting count. The closest is marked when the scores differ, and tapping one switches the details to it.
Alternate flows	2a) The request fails: an error with “Try again”. 2b) The profile names a course, skill or career the API no longer serves: it is left out, and a dropped main career gives way to the next.
Postcondition	Nothing is stored; the reading is computed for each request

UC-3 Plan courses (what-if)

Field	Description
Actor	Student
Precondition	A profile is saved
Main flow	1) The student opens “Plan courses” from home or readiness. 2) The app lists the courses not yet taken, by track, each with the current gaps it would close. 3) The student ticks a course, and the plan is saved on the device at once. 4) The app calls POST /gap again with completed and planned courses together. 5) The projection stands beside today's figure, with the points gained and the gaps the plan closes, each counted once.

Alternate flows	1a) With several careers, the student switches the career the plan is read against. 3a) The save fails: the plan is put back and a message says so.
Postcondition	The planned courses are saved with the profile; a course later marked completed leaves the plan

UC-4 View suggested courses

Field	Description
Actor	Student; the web browser for the extension
Precondition	A profile is saved
Main flow	1) From readiness, the student taps “Suggested courses”. 2) The app sends the profile to POST /recommendations. 3) University courses come first, ranked by the demand they close, each marked as counting toward the degree. 4) Online courses follow, only for skills no catalog course teaches, each marked as not counting. 5) Missing skills with neither are listed, not dropped.
Extension	The student taps an online course and the browser opens it; if nothing can open it, a message says so
Postcondition	Nothing is stored

UC-5 View curriculum gaps

Field	Description
Actor	Department staff
Precondition	None; the department tab works with or without a student profile
Main flow	1) Staff open the “Department” tab, which is built the first time it opens. 2) The sample-size caveat comes first, with the real posting counts, then the quality flag. 3) GET /curriculum/gap returns coverage (taught skills over demanded skills) and the threshold in postings. 4) The screen lists demanded skills no course teaches, by demand, with the careers asking; skills named by too few postings to count; taught skills no posting names; and coverage for each career.
Extension	Staff pick one career's pill to see that career alone
Postcondition	Nothing is stored

3. Activity diagrams

Four activities carry the system: the student's journey through the app, the gap calculation behind every readiness figure, the engine's start-up, and what-if planning.

3a. The student's journey

The app decides at launch: no saved profile means onboarding, and a saved one goes straight to home. Every reading after that is a round trip to POST /gap, and every option from home leads back to it.

Activity diagram: a student's journey, from first launch to the readiness reading

Swimlanes show who acts. Diamonds are decisions and merges; guards are in square brackets. Connector A joins the two ends of the loop back to home.

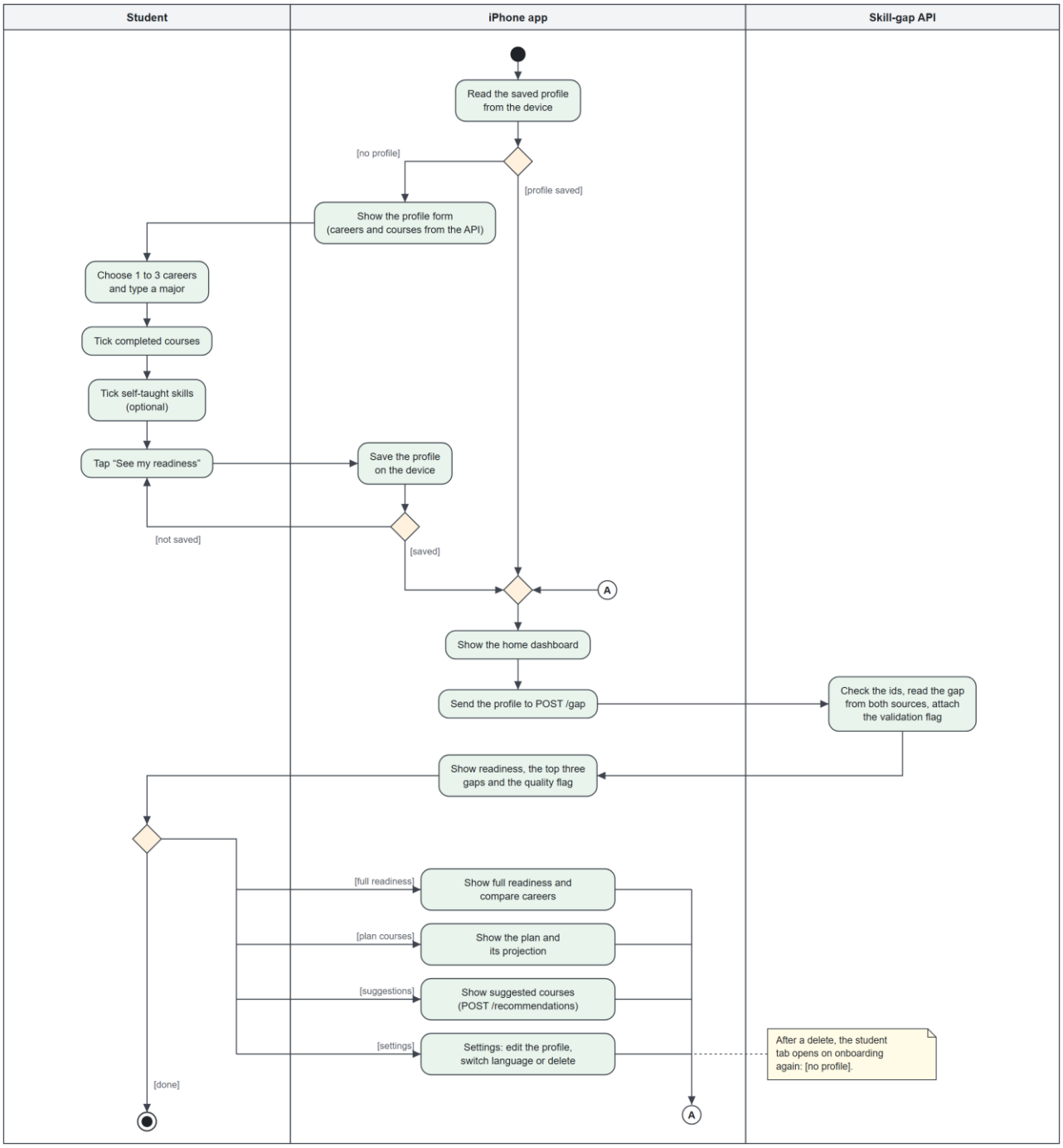


Figure 6. Activity diagram of the student's journey through the app

3b. The gap calculation (POST /gap)

The request is checked before anything is computed, and an unknown career, course or skill ends it with HTTP 422. The gap is then read twice, independently, once with self-reported skills and once from courses alone, so the academic record is never blurred with the student's own claim.

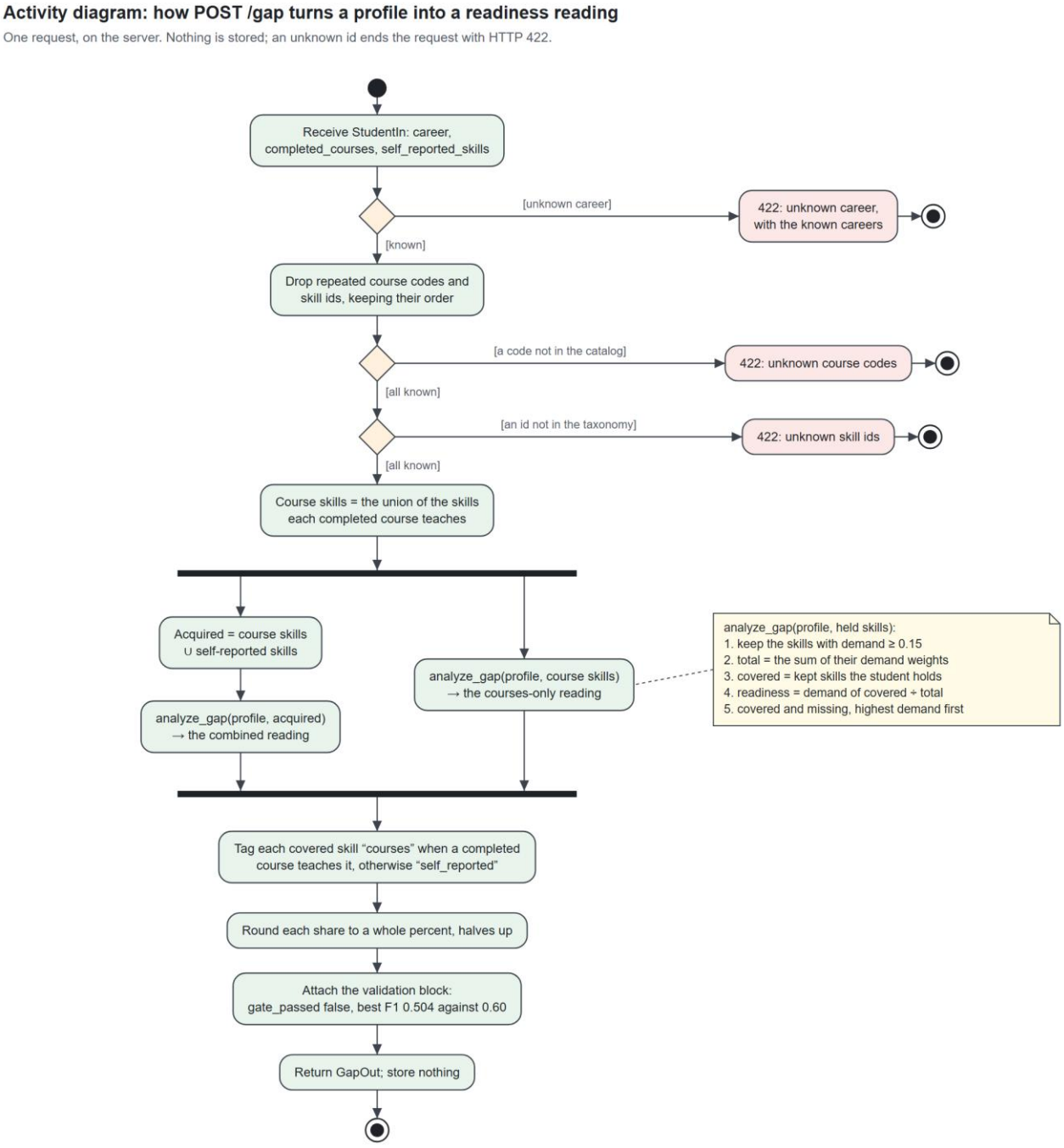


Figure 7. Activity diagram of the POST /gap calculation

3c. Engine start-up

The server refuses to start on a stale config or a bad label rather than serve wrong figures. When the cache key matches, start-up skips the model entirely; only a changed input brings back the embedding work.

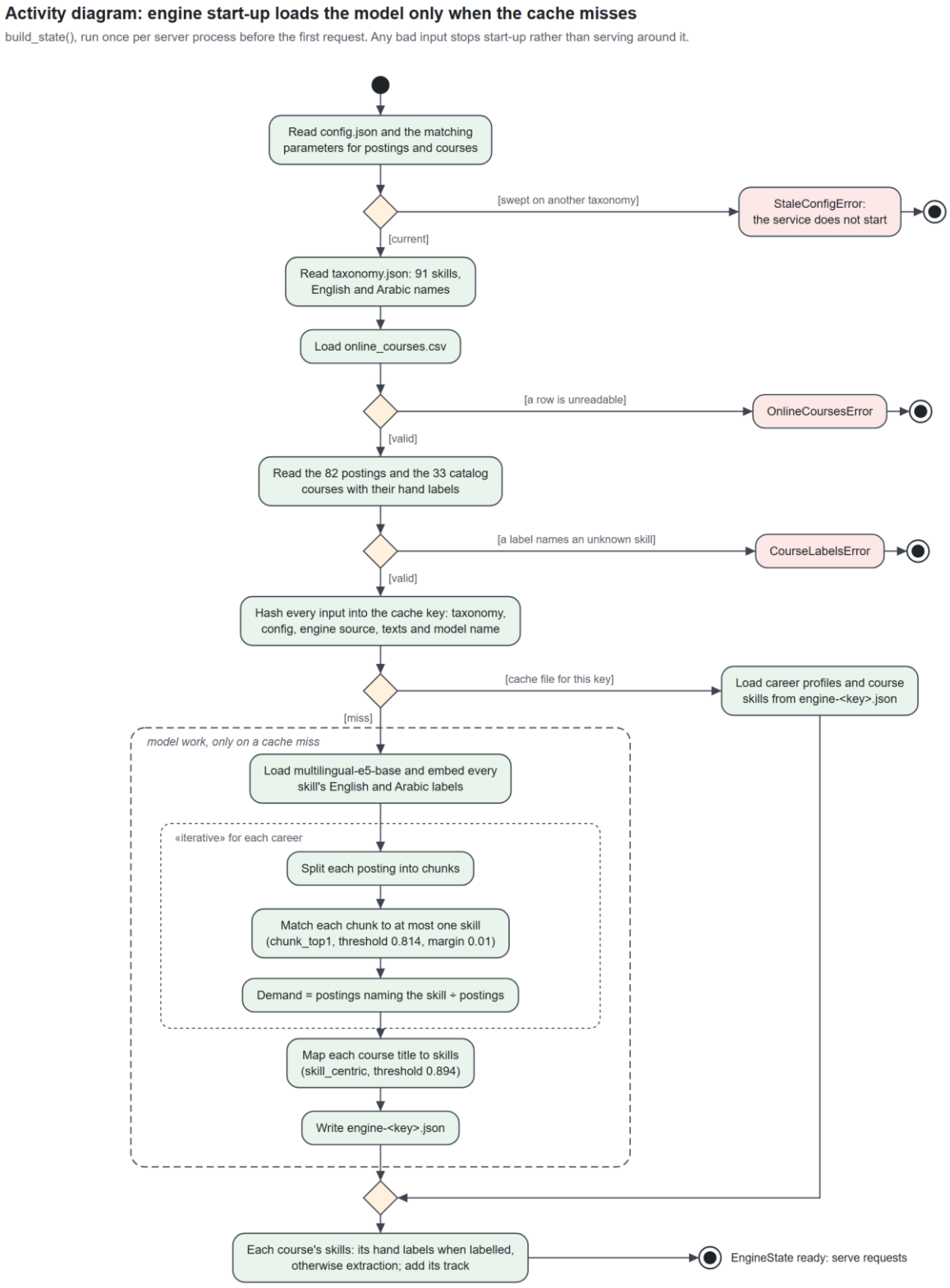


Figure 8. Activity diagram of the engine start-up and profile building

3d. What-if planning

A tick shows on screen before the save completes, and a failed save puts the plan back. The projection is the same gap calculation with planned courses counted as taken.

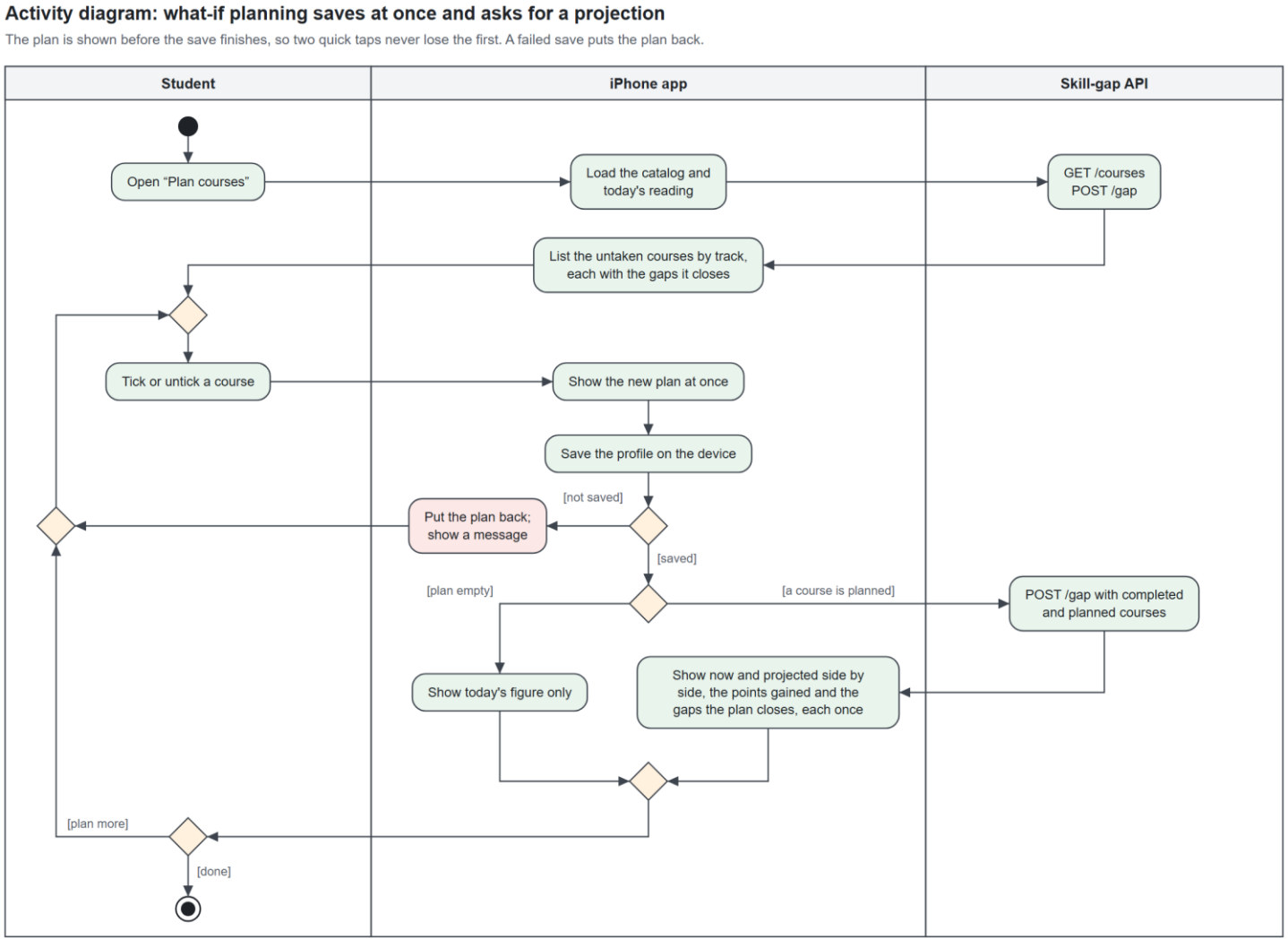


Figure 9. Activity diagram of what-if course planning

4. Sequence diagrams

Six interactions, traced through the real classes and endpoints. Filled arrowheads are calls, dashed arrows are replies, and the bars show which object is working.

4a. Onboarding and saving the profile

The form's lists come from three GET requests. Finishing writes one JSON document to the phone's shared_preferences; nothing goes to the server.

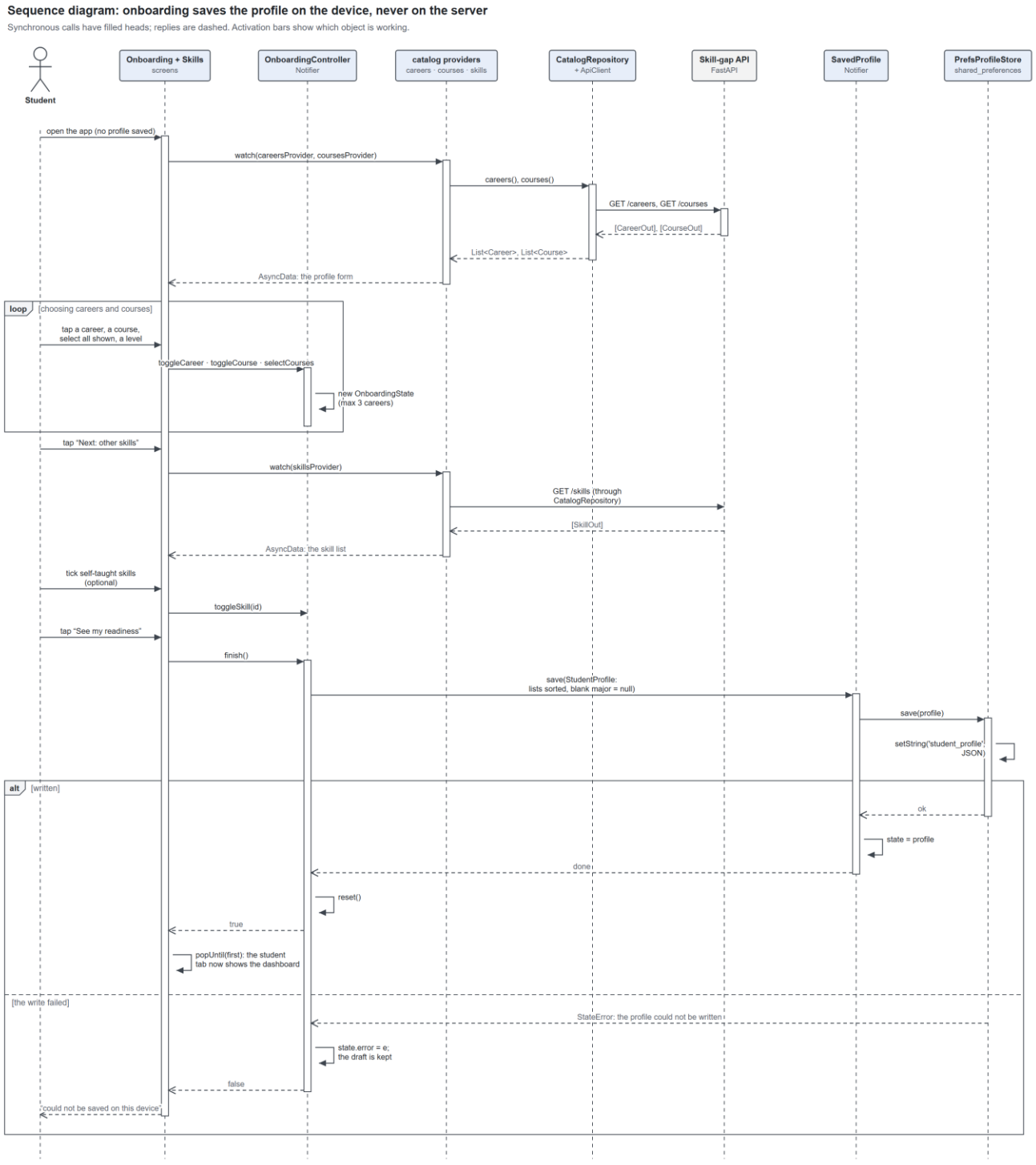


Figure 10. Sequence diagram of onboarding and saving the profile

4b. Reading readiness on home

The dashboard first drops any id the API no longer serves, then posts the profile to /gap. The engine reads the gap twice, with self-reported skills and with courses alone.

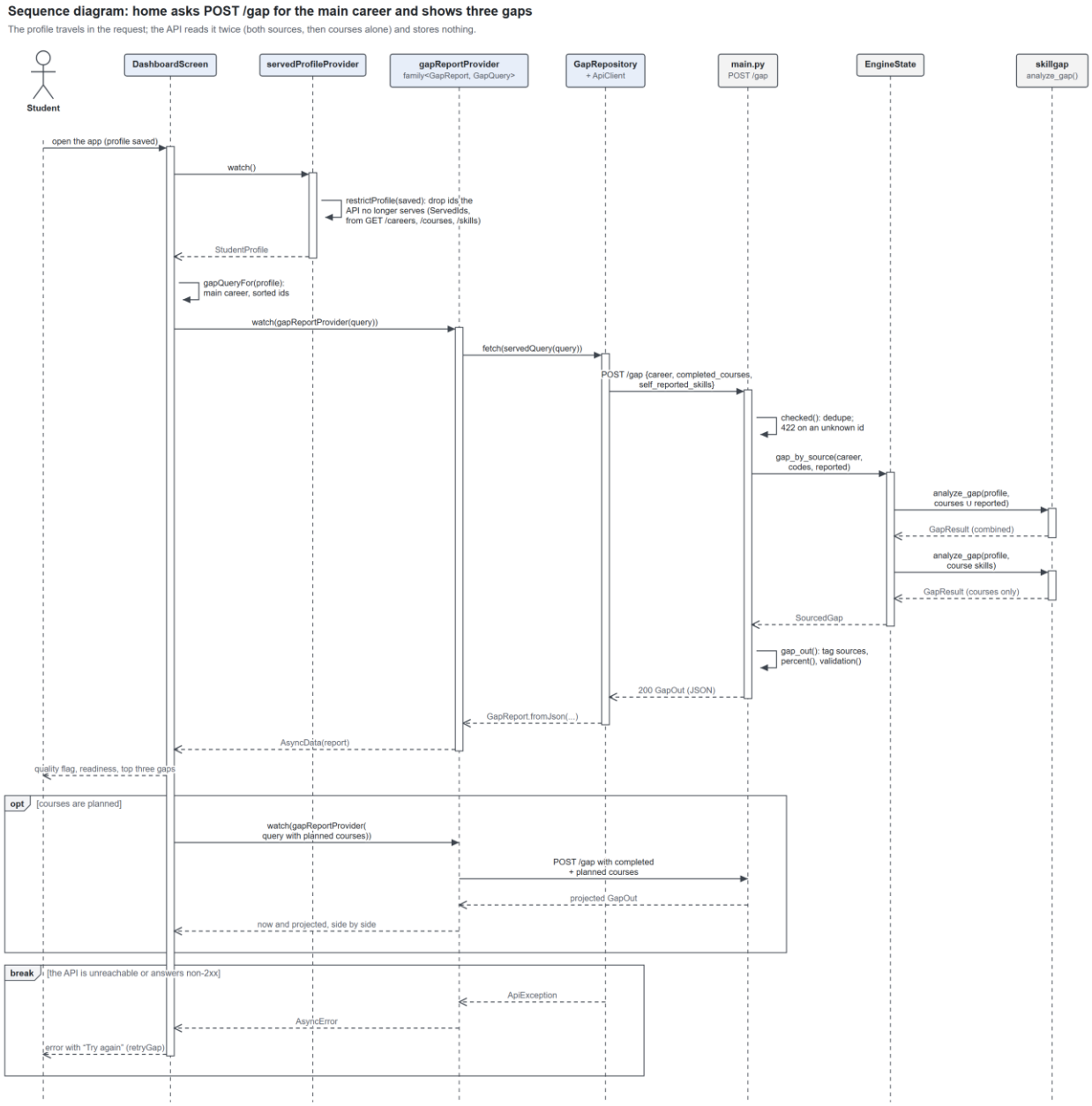


Figure 11. Sequence diagram of reading readiness on the home dashboard

4c. Suggested courses

University courses that close a gap come first and count toward the degree. Online courses are offered only for gaps no catalog course teaches, and open in the phone's browser.

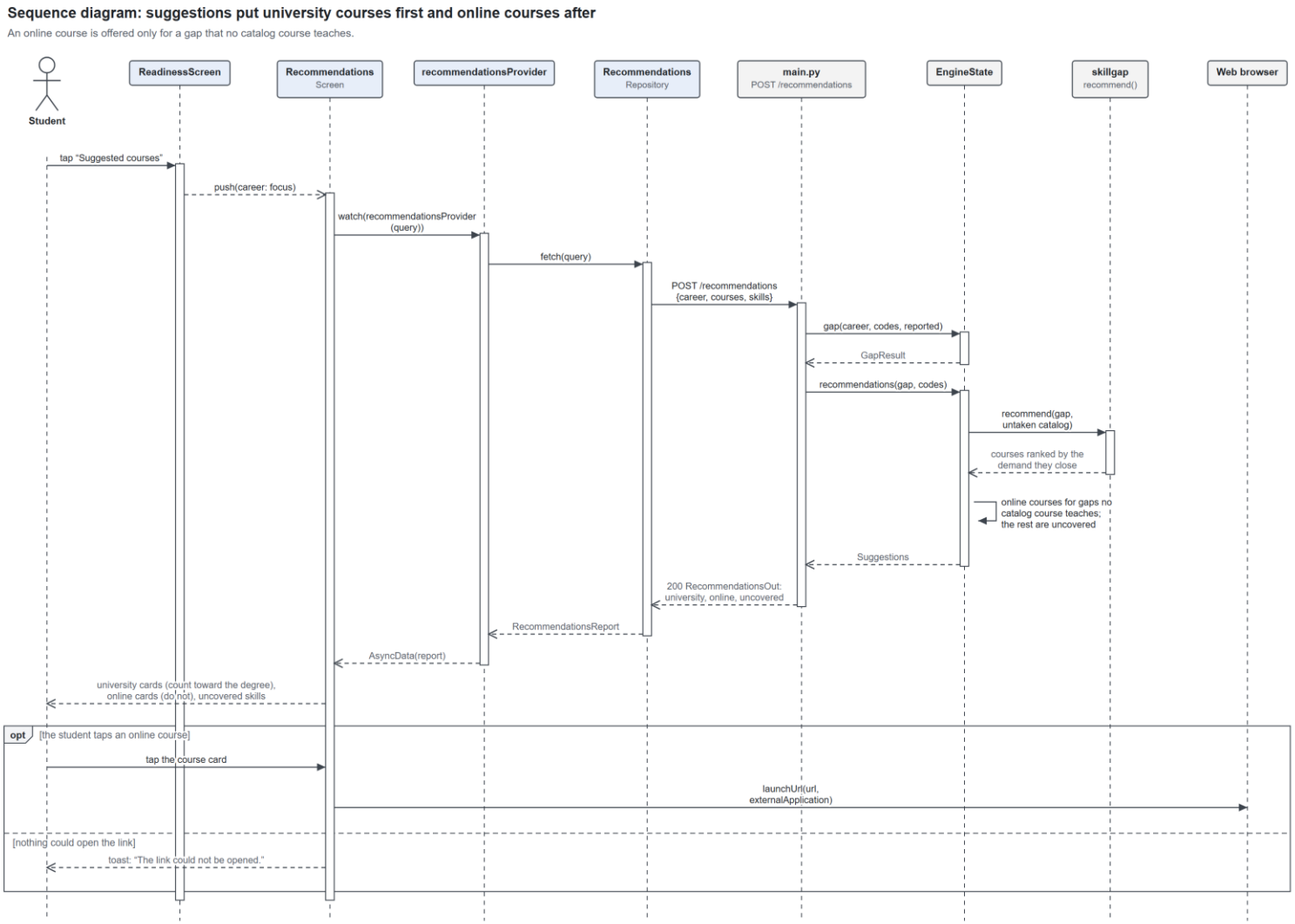


Figure 12. Sequence diagram of the suggested courses

4d. Planning a course

The plan changes on screen before the save finishes, so a second quick tap builds on the first; a failed save rolls it back. The projection is a second POST /gap with planned courses counted as taken.

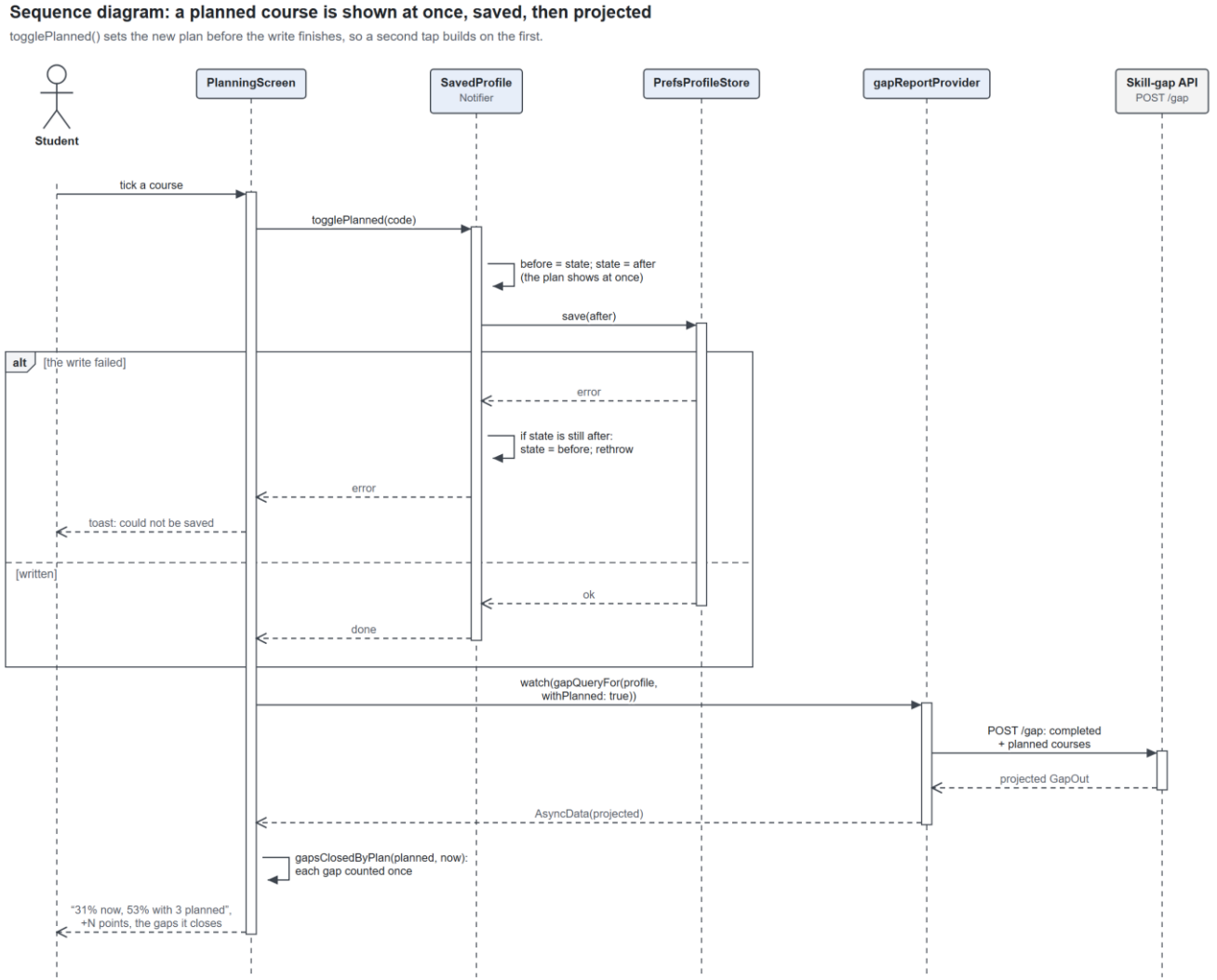


Figure 13. Sequence diagram of planning a course

4e. The department view

The tab is built the first time it opens. Every career's view comes first, and a career pill repeats the same request for one career.

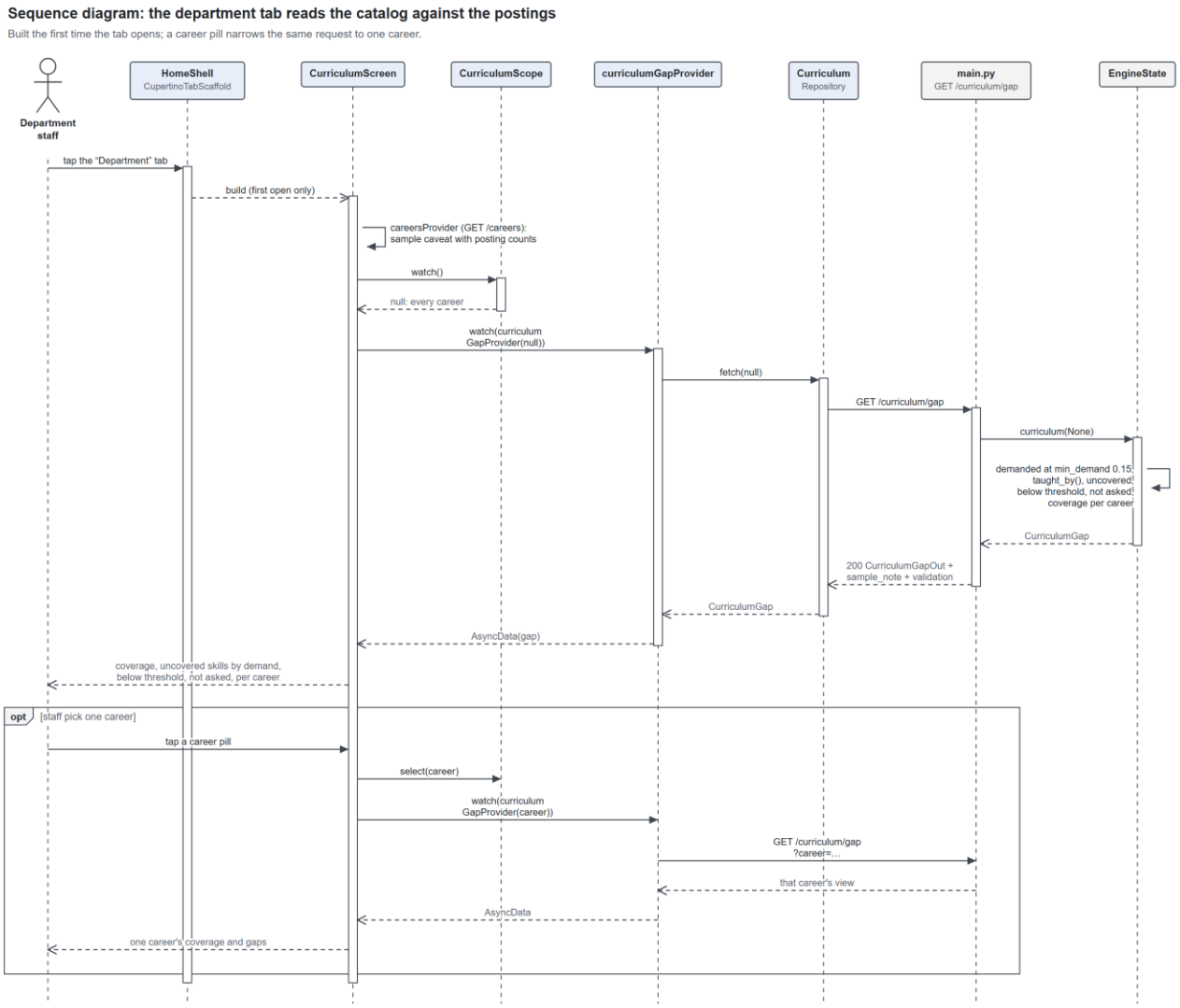


Figure 14. Sequence diagram of the department's curriculum view

4f. Server start-up

The engine is built once per process. When the cache file matches the key, the model is never created; on a miss it is downloaded once, and the results are written back to the cache.

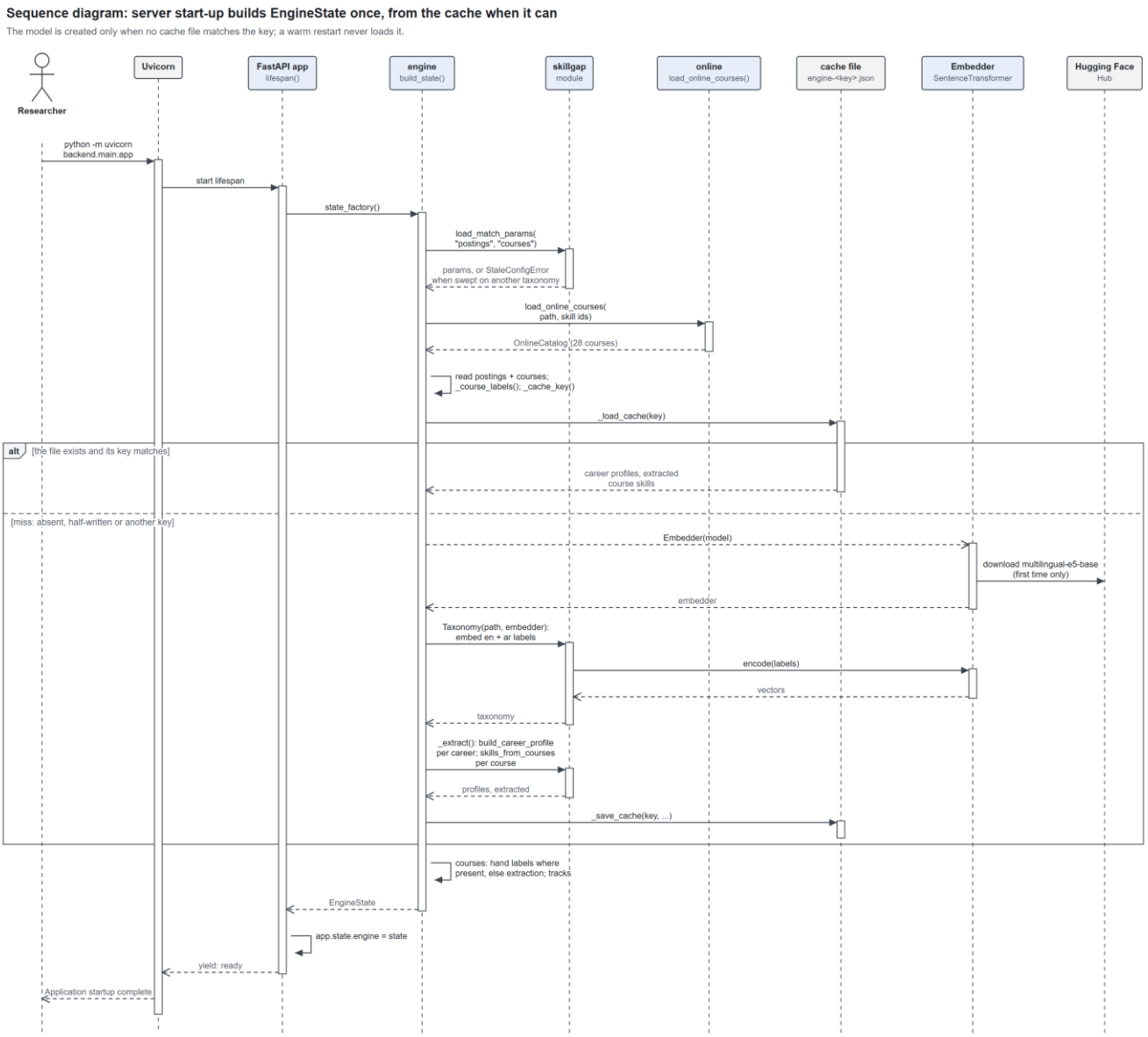


Figure 15. Sequence diagram of the server start-up

5. Database diagram

The system has no database server. The data is nine stores: eight CSV and JSON files on the server, six of which the engine reads once at start-up, and a key-value store on the phone for the student's profile. The server's SQLite profile database was removed on 1 October 2026, so no student data is kept on the server; this is deliberate, and accounts are future work.

Store	Where	Rows	Written by	Read by
taxonomy.json	server, skillgap/	91 skills	hand	engine, validate.py
config.json	server, skillgap/	1 document	validate.py	engine start-up
labeled_postings.csv	server, test/	82 postings, 15 labelled	collect_postings.py, hand labels	engine, validate.py
labeled_postings_excluded.csv	server, test/	28 postings	collect_postings.py	the researcher
labeled_courses.csv	server, skillgap/	33 courses	hand	engine, validate.py
labeled_courses_excluded.csv	server, skillgap/	15 courses	hand	the researcher
online_courses.csv	server, skillgap/	47 rows, 28 courses	hand; online.py sync adds placeholders	engine
engine-<key>.json	server, backend/.cache/	8 profiles, 33 courses	engine, on a cache miss	engine start-up
shared_preferences	the iPhone	2 keys	the app	the app

The skill lists inside the CSV files are stored as one field of ids separated by “;”. Normalised into tables, each list would become a link table between its store and the taxonomy; the ER diagram in section 7 shows those relationships.

5a. The data stores

Each box is a store and its fields. PK identifies a row, FK points at another store, and the crow's feet give each link's cardinality.

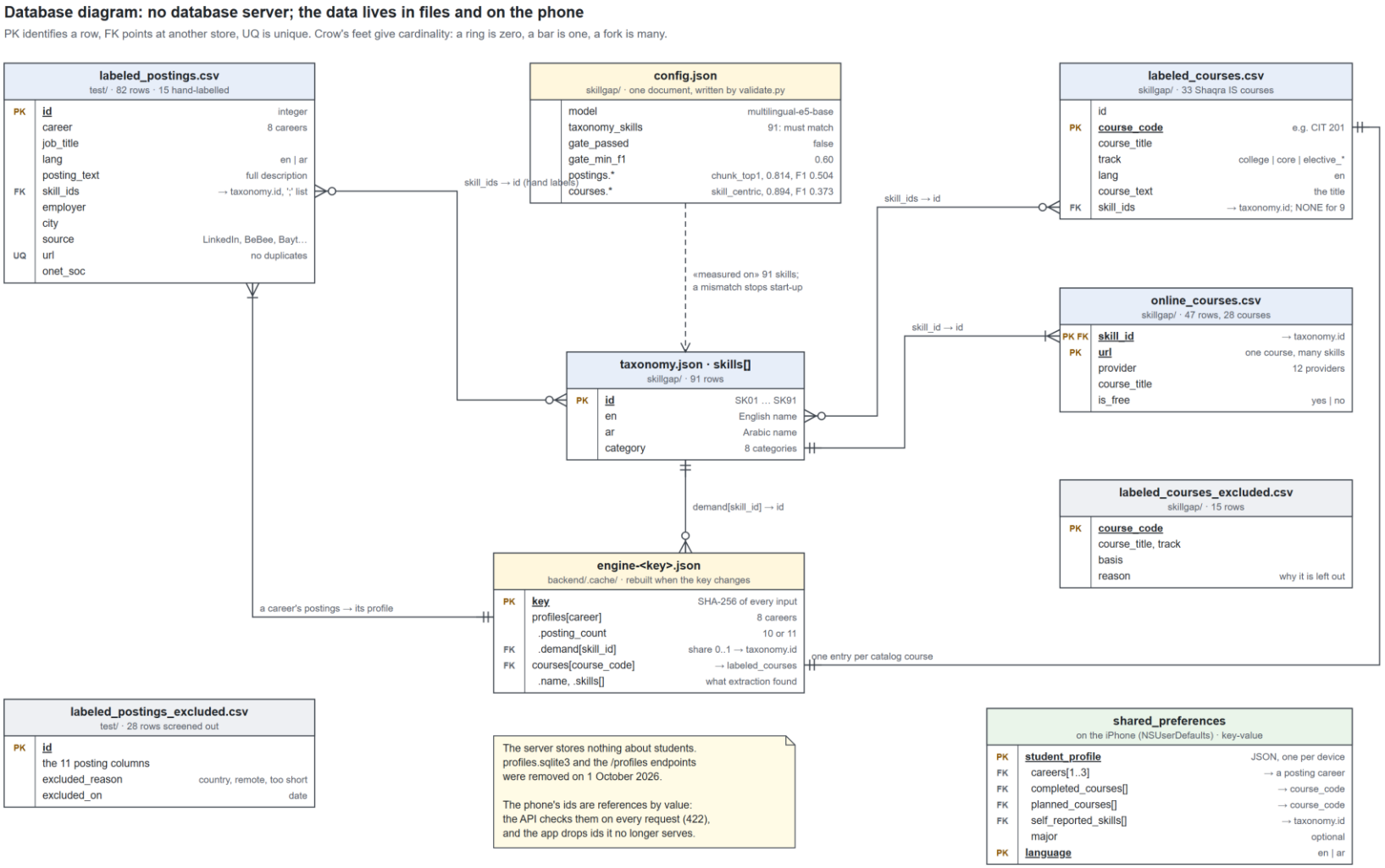


Figure 16. Database diagram of the data files and the phone's key-value store

6. Class diagrams

Five class diagrams cover the code, from the engine to the screens. Names, fields and signatures are taken from the source; + is public, - is private, a filled diamond is composition, a hollow triangle is generalisation, and a dashed arrow is a dependency.

6a. The skill-gap engine (skillgap.py)

The Taxonomy embeds every skill's English and Arabic labels with one Embedder. The module's functions turn postings into a CareerProfile and a student's skills into a GapResult.

Class diagram: the skill-gap engine (skillgap/skillgap.py)

The matcher, the taxonomy it embeds, and the two results the service reads: a career's demand profile and a student's gap.

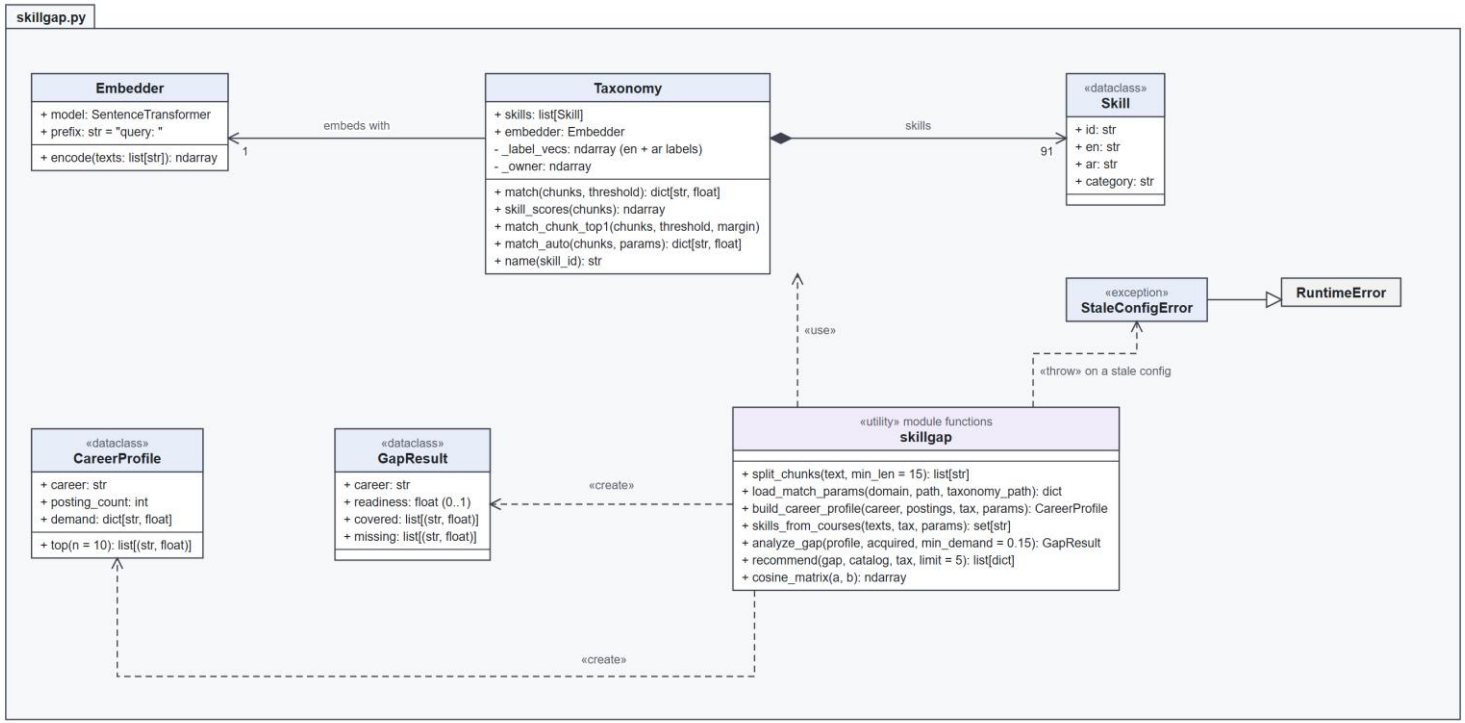


Figure 17. Class diagram of the skill-gap engine

6b. The service (engine.py, online.py, main.py)

One EngineState holds the 33 courses, 8 career profiles and the online catalog. Every response is built from it, as a SourcedGap, Suggestions or CurriculumGap.

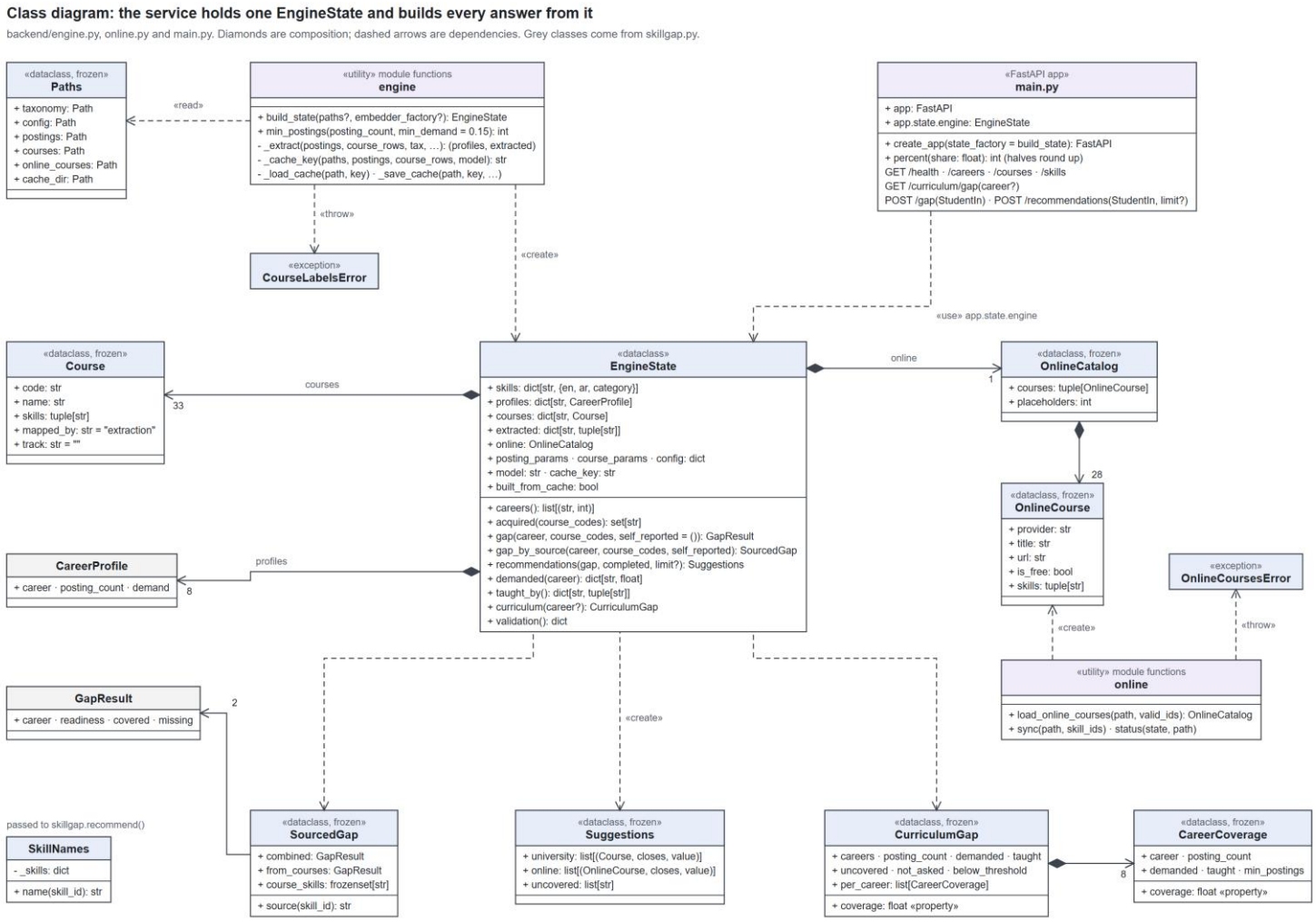


Figure 18. Class diagram of the service's EngineState and its parts

6c. The API's request and response objects (schemas.py)

StudentIn is the only request body. Each response that reports a figure carries ValidationInfo, and a recommendation is one of two shapes told apart by its source field.

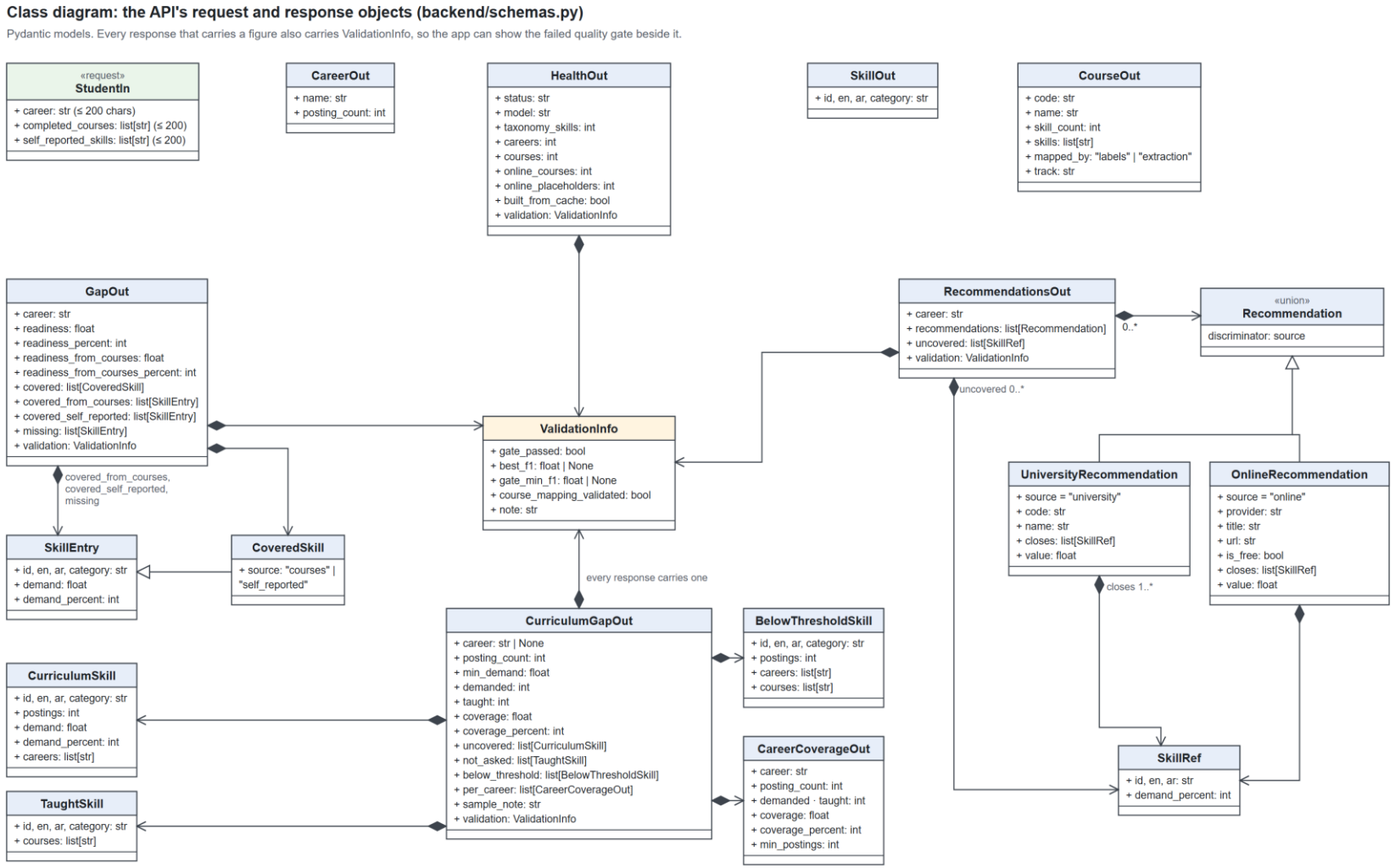


Figure 19. Class diagram of the API's request and response objects

6d. The app's domain model

The app's models mirror the API's responses in camelCase, keeping the fields the screens use. StudentProfile is the only one the app writes; ServedIds trims a saved profile to what the API still serves.

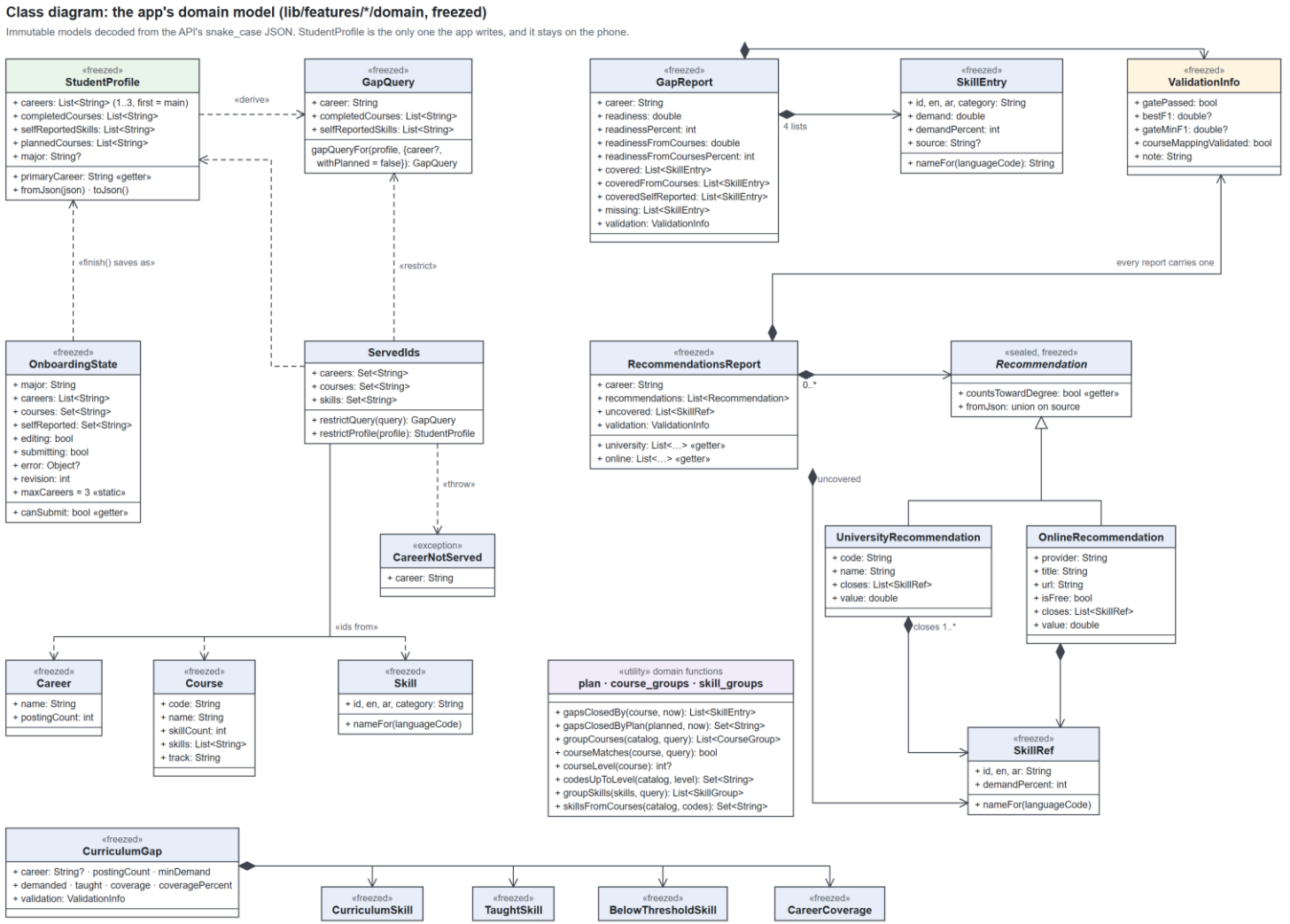


Figure 20. Class diagram of the app's domain model

6e. The app's layers

Screens watch Riverpod notifiers and providers; providers call repositories, and repositories call one ApiClient. The profile goes through the ProfileStore interface, which the phone's shared_preferences implements and the tests replace with a fake.

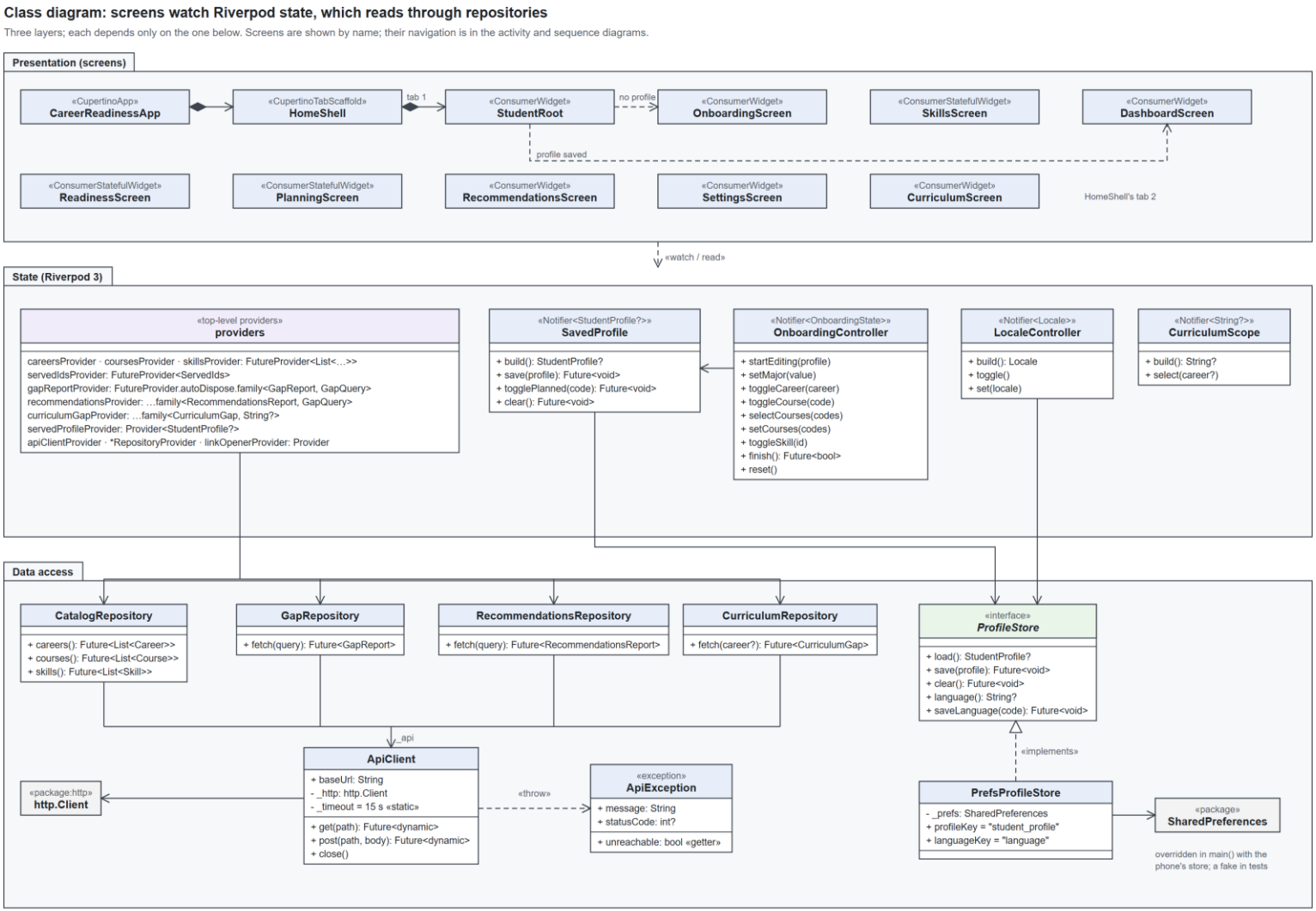


Figure 21. Class diagram of the app's screens, state and data access

7. ER diagram

SKILL is the hub of the model: postings, careers, courses, online courses and the student all connect through it. There are seven entities and ten relationships, all many-to-many except a posting's career and a skill's category.

7a. Entities and relationships

Rectangles are entities, diamonds relationships and ovals attributes; an underlined attribute is a key, a dashed one is derived, and a double line means every instance takes part.

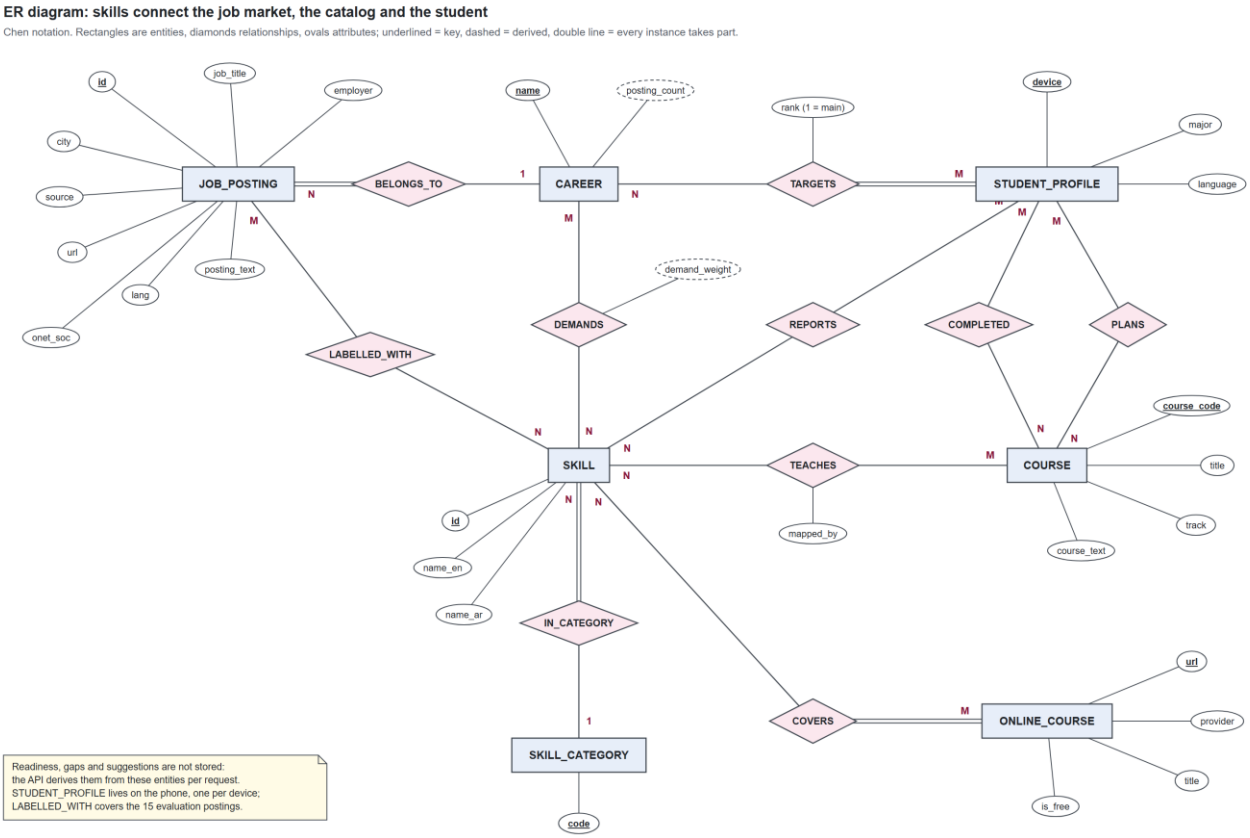


Figure 22. ER diagram in Chen notation

7b. Relationships

Relationship	Between	Cardinality	Meaning
BELONGS_TO	JOB_POSTING, CAREER	N : 1	Every posting is collected for one career
DEMANDS	CAREER, SKILL	M : N	A career's demand for a skill: the share of its postings naming it (derived)
LABELLED_WITH	JOB_POSTING, SKILL	M : N	Hand labels, on the 15 evaluation postings only
IN_CATEGORY	SKILL, SKILL_CATEGORY	N : 1	Each of the 91 skills is in one of 8 categories
TEACHES	COURSE, SKILL	M : N	The served course mapping; mapped_by is labels or extraction
COVERS	ONLINE_COURSE, SKILL	M : N	An online course offered for a skill no catalog course teaches
TARGETS	STUDENT_PROFILE, CAREER	M : N	1 to 3 careers per student; rank 1 is the main career
COMPLETED	STUDENT_PROFILE, COURSE	M : N	Courses the student has taken
PLANS	STUDENT_PROFILE, COURSE	M : N	Courses in the what-if plan
REPORTS	STUDENT_PROFILE, SKILL	M : N	Skills the student holds from outside coursework; unverified

List of figures

Figure 1. Component diagram of the app, the API, the engine and the data files

Figure 2. Deployment diagram of the phone, the server and the outside services

Figure 3. Package diagram of the app's feature packages

Figure 4. Use case diagram for students and department staff

Figure 5. Use case diagram for the researcher's data-preparation tools

Figure 6. Activity diagram of the student's journey through the app

Figure 7. Activity diagram of the POST /gap calculation

Figure 8. Activity diagram of the engine start-up and profile building

Figure 9. Activity diagram of what-if course planning

Figure 10. Sequence diagram of onboarding and saving the profile

Figure 11. Sequence diagram of reading readiness on the home dashboard

Figure 12. Sequence diagram of the suggested courses

Figure 13. Sequence diagram of planning a course

Figure 14. Sequence diagram of the department's curriculum view

Figure 15. Sequence diagram of the server start-up

Figure 16. Database diagram of the data files and the phone's key-value store

Figure 17. Class diagram of the skill-gap engine

Figure 18. Class diagram of the service's EngineState and its parts

Figure 19. Class diagram of the API's request and response objects

Figure 20. Class diagram of the app's domain model

Figure 21. Class diagram of the app's screens, state and data access

Figure 22. ER diagram in Chen notation